

NAME: ANANYA MANISH JADHAV

CLASS: A

BATCH: A4

PRN: 202401040097

ROLL NO: 62

THREADED BINARY TREE

CODE :-

```
#include <iostream>
```

```
#include <stack>
```

```
using namespace std;
```

```
struct Node
```

```
{
```

```
    int data;
```

```
    Node *left, *right;
```

```
    bool lthread, rthread;
```

```
};
```

```
Node* root = NULL;
```

```
// Create Node
```

```
Node* createNode(int val)
```

```
{
```

```
    Node* temp = new Node;
```

```
temp->data = val;

temp->left = temp->right = NULL;

temp->lthread = temp->rthread = true;

return temp;

}
```

// Insert in Threaded Binary Tree

```
void insert(int val)

{

    Node *ptr = root;

    Node *par = NULL;

    while(ptr != NULL)

    {

        if(val == ptr->data)

        {

            cout<<"Duplicate value\n";

            return;

        }

        par = ptr;

        if(val < ptr->data)

        {

            if(ptr->lthread == false)
```

```
        ptr = ptr->left;
    else
        break;
}
else
{
    if(ptr->rthread == false)
        ptr = ptr->right;
    else
        break;
}
}
```

```
Node *temp = createNode(val);
```

```
if(par == NULL)
```

```
    root = temp;
```

```
else if(val < par->data)
```

```
{
```

```
    temp->left = par->left;
```

```
    temp->right = par;
```

```
    par->lthread = false;
```

```
    par->left = temp;
```

```
}  
  
else  
  
{  
  
    temp->left = par;  
  
    temp->right = par->right;  
  
    par->rthread = false;  
  
    par->right = temp;  
  
}  
  
}
```

```
Node* leftMost(Node* ptr)  
{  
  
    while(ptr->lthread == false)  
  
        ptr = ptr->left;  
  
    return ptr;  
  
}
```

// Inorder Traversal (Non Recursive)

```
void inorder()  
  
{  
  
    Node* ptr = leftMost(root);  
  
  
    while(ptr != NULL)  
  
    {
```

```
cout<<ptr->data<<" ";

if(ptr->rthread)

    ptr = ptr->right;

else

{

    ptr = ptr->right;

    ptr = leftMost(ptr);

}

}
```

// Preorder Traversal

```
void preorder(Node* root)

{

    if(root == NULL) return;

    cout<<root->data<<" ";

    if(!root->lthread)

        preorder(root->left);

    if(!root->rthread)

        preorder(root->right);

}
```

// Postorder Traversal

```
void postorder(Node* root)
```

```
{
```

```
    if(root == NULL) return;
```

```
    if(!root->lthread)
```

```
        postorder(root->left);
```

```
    if(!root->rthread)
```

```
        postorder(root->right);
```

```
    cout<<root->data<<" ";
```

```
}
```

```
// Non Recursive Preorder
```

```
void preorderNR()
```

```
{
```

```
    if(root==NULL) return;
```

```
    stack<Node*> s;
```

```
    s.push(root);
```

```
    while(!s.empty())
```

```
    {
```

```
        Node* temp = s.top();
```

```
        s.pop();
```

```
cout<<temp->data<<" ";
```

```
if(!temp->rthread)
```

```
    s.push(temp->right);
```

```
if(!temp->lthread)
```

```
    s.push(temp->left);
```

```
}
```

```
}
```

```
// Non Recursive Postorder
```

```
void postorderNR(Node* root)
```

```
{
```

```
    if(root == NULL) return;
```

```
    stack<Node*> s1, s2;
```

```
    s1.push(root);
```

```
    while(!s1.empty())
```

```
{
```

```
    Node* temp = s1.top();
```

```
    s1.pop();
```

```
    s2.push(temp);
```

```
    if(!temp->lthread)
```

```
        s1.push(temp->left);
```

```

        if(!temp->rthread)
            s1.push(temp->right);
    }

    while(!s2.empty())
    {
        cout<<s2.top()->data<<" ";
        s2.pop();
    }
}

int main()
{
    int ch,val;

    while(true)
    {
        cout<<"\n1.Insert\n2.Inorder\n3.Preorder\n4.Postorder\n5.Preorder
NonRec\n6.Postorder NonRec\n7.Exit\n";

        cin>>ch;

        switch(ch)
        {
            case 1:
                cout<<"Enter value:";

                cin>>val;

```


insert(val);

break;

case 2:

inorder();

break;

case 3:

preorder(root);

break;

case 4:

postorder(root);

break;

case 5:

preorderNR();

break;

case 6:

postorderNR(root);

break;

case 7:

return 0;

}

}

}

OUTPUT:- 1.Insert

2.Inorder Traversal

3.Preorder Traversal

4.Recursive Inorder

5.Recursive Preorder

6.Recursive Postorder

7.Exit

Enter choice: 1

Enter value: 45

Enter choice: 1

Enter value: 15

Enter choice: 1

Enter value: 79

Enter choice: 1

Enter value: 10

Enter choice: 1

Enter value: 20

Enter choice: 1

Enter value: 55

Enter choice: 1

Enter value: 90

Enter choice: 2

Inorder Traversal:

10 15 20 45 55 79 90

Enter choice: 3

Preorder Traversal:

45 15 10 20 79 55 90

Enter choice: 4

Recursive Inorder:

10 15 20 45 55 79 90

Enter choice: 5

Recursive Preorder:

45 15 10 20 79 55 90

Enter choice: 6

Recursive Postorder:

10 20 15 55 90 79 45